

Machine Learning Homework 5 Report

314540075 Shalini Fatehpuriya (方莎琳)

Gaussian Process:

Goal: The goal of this implementation is to use Gaussian Process Regression (GPR) to learn an unknown function from noisy training data and predict its behavior over a continuous range of input values.

I. Part 1:

1. **Loading The data:** The training dataset is loaded from the input file where each row contains one training sample (x_i, y_i) . The observed target values are assumed to follow:

$$Y_i = f(x_i) + \epsilon; \text{ where } \epsilon \sim \mathcal{N}(0, \beta^{-1})$$

```
def createMatrix(filename):
    data = [[0.0 for _ in range(2)] for _ in range(34)]
    with open(filename, "r") as file:
        for i in range(34):
            values = file.readline().split()
            data[i][0] = float(values[0])
            data[i][1] = float(values[1])
    return data
```

2. **Calculating the Rational Quadratic Kernel:** The Rational Quadratic Kernel computes similarity between two input points and defines the covariance structure of the Gaussian Process. The kernel formula is:

Rational Quadratic
$\tilde{K}(x_1 - x_2) = \sigma^2 \left(1 + \frac{(x_1 - x_2)^2}{2\alpha l^2} \right)^{-\alpha}$

The code Implementing the above formula is:

```
def rationalQuadratic(x1, x2, alpha, l, sigma):
    diff = x1 - x2
    sqdist = diff * diff
    base = 1.0 + (sqdist / (2.0 * alpha * l * l))
    return (sigma * sigma) * (base ** (-alpha))
```

3. **Calculating the Covariance Matrix:** The covariance matrix KKK is constructed using the Rational Quadratic Kernel for all pairs of training points. Noise variance is added to diagonal elements to model observation noise.

The covariance matrix is defined as:

$$K_{ij} = k(x_i, x_j) + \beta^{-1} \delta_{ij}$$

Given the value β of is taken as 5.

```
def buildKernel(X, alpha, l, sigma, beta):
    n = len(X)
    K = [[0.0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            K[i][j] = rationalQuadratic(X[i], X[j], alpha, l, sigma)
            if i == j:
                K[i][j] += 1.0 / beta
    return K
```

4. Gaussian Prediction before Optimisation:

a. Computing Mean, Variance and Negative Marginal Log Likelihood:

- i. We first take the initial values of Beta, Alpha l and sigma square. Gaussian Process Regression predicts both the mean and uncertainty of the unknown function.

The predictive mean is computed as:

$$\mu(x_*) = k_*^T K^{-1} Y$$

The predictive variance is computed as:

$$\sigma^2(x_*) = k(x_*, x_*) - k_*^T K^{-1} k_*$$

The 95% confidence interval is:

$$\mu(x_*) \pm 1.96\sigma(x_*)$$

All the values of X_i mean and 95% confidence interval (both + and -) are stored in the “output_before.txt” file.

The Plot will be plotted once both Before and After points are accumulated.

Post the prediction the NMLL is calculated using the below function.

$$-\log p(Y|X) = \frac{1}{2} \log |K| + \frac{1}{2} Y^T K^{-1} Y + \frac{n}{2} \log(2\pi)$$

```
def computeNMLL(K, K_inv, Y):
    n = len(Y)
    detK = determinant([row[:] for row in K])
    logDet = math.log(abs(detK) + 1e-12)

    temp = matVecMul(K_inv, Y)
    quad = dot(Y, temp)
    constant = (n / 2.0) * math.log(2.0 * math.pi)
    return 0.5 * logDet + 0.5 * quad + constant
```

II. Part 2;

Later the parameters are optimised using the below function. Here the “**minimize**” of the **scipy** library is being used.

Main:

```
def objective(theta, X, Y, beta):
    alpha = theta[0]
    l = theta[1]
    sigma = theta[2]

    K = buildKernel(X, alpha, l, sigma, beta)
    K_inv = invertMatrix([row[:] for row in K])

    return computeNMLL(K, K_inv, Y)

result = minimize(objective,[alpha, l, sigma],args=(X, Y, beta),method='L-BFGS-B',bounds=
[(1e-5, None), (1e-5, None), (1e-5, None)])
alpha, l, sigma = result.x

print("Optimized parameters:")
print("alpha =", alpha, "l =", l, "sigma =", sigma)
```

Once we have the optimised parameters we again compute the NMLL using the same function which was used before.

The output is now again stored in the “output_after.txt” file.

```
K_after = buildKernel(X, alpha, l, sigma, beta)
Kinv_after = invertMatrix([row[:] for row in K_after])

GPTestPointsPredict(Kinv_after, X, Y, 200,alpha, l, sigma, beta,"./output/output_after.txt")
print("After NLML =", computeNMLL(K_after, Kinv_after, Y))
```

The Graphs are plotted using “output_before.txt” and output_after.txt” using below function from numPy and PyPlot

```
plot_gp("./output/output_before.txt","./output/train.txt","./output/gp_result.jpg")
plot_gp("./output/output_after.txt","./output/train.txt","./output/gpOpt_result.jpg")
```

```
def plot_gp(before_file, train_file, output_image):
    # data loading
    train = np.loadtxt(train_file)
    X_train = train[:, 0]
    Y_train = train[:, 1]
    before = np.loadtxt(before_file)
    x = before[:, 0]
    mean = before[:, 1]
    upper = before[:, 2]
    lower = before[:, 3]

    # plotting
    plt.figure(figsize=(12, 8))
    # 95% confidence interval
    plt.fill_between(x, lower, upper, color="lightblue", label="95% Confidence Interval")
    # mean line
    plt.plot(x, mean, color="blue", linewidth=2, label="Mean")
    # training points
    plt.scatter(X_train, Y_train, color="black", s=60, label="Training Data")
    # titles legends
    plt.title("Gaussian Process Regression")
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.grid(True)
    plt.legend()

    # saving
    plt.savefig(output_image, dpi=300, bbox_inches="tight")
    plt.close()

    print("Saved:", output_image)
```

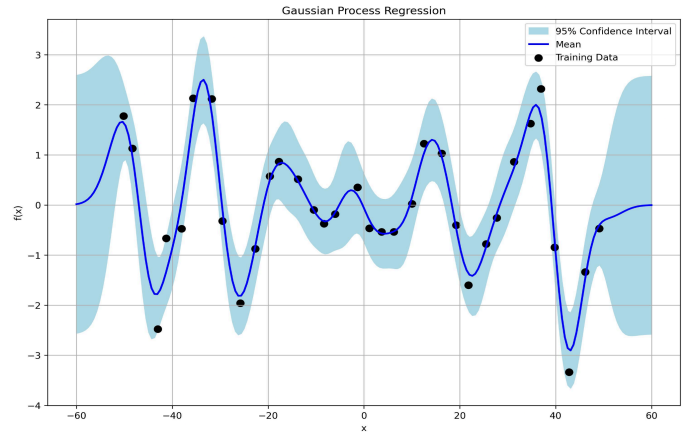
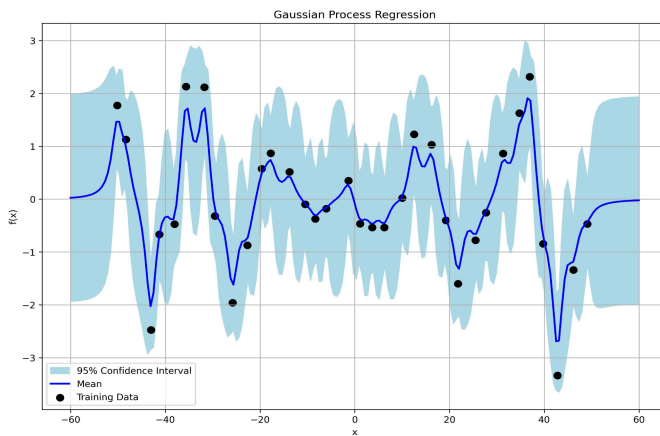
5. Results:

Before Optimisation:

$$\alpha = 1, \lambda = 1, \sigma^2 = 1, \beta = 5$$

After Optimisation:

$$\alpha = 3542.59, \lambda = 3.31, \sigma^2 = 1.31, \beta = 5$$



6. Conclusion:

- a. Before optimization, the prediction curve was more noisy and the confidence interval was wider. After optimization, the model fit the data more smoothly and produced more stable confidence regions.
- b. α controls how smooth the Gaussian Process curve looks. Smaller values make the curve more irregular, while larger values make it smoother. After optimization, α became very large, so the prediction curve became much smoother.
- c. σ^2 controls the overall variation of the predicted function. Higher values allow the model to capture larger changes in the data. (β) represents the noise level in the observations. Since β was fixed, the assumed noise level stayed the same during optimization.

SVM on MNIST dataset:

The goal of this implementation is to classify handwritten digit images (digits 0–4) using Support Vector Machine (SVM) classifiers with different kernel functions. Each image is represented as a 28×28 grayscale image, which is flattened into a 784-dimensional feature vector. The SVM model learns a decision boundary that separates different digit classes with maximum margin.

I. Part 1: Comparison b/w Linear, RBF and Polynomial:

1. **Loading the Data:** The data is loaded using python's pandas library. Post loading the data was normalised to avoid feature dominance using min. And max. Values.

```
def load_data(x_path, y_path):
    X = pandas.read_csv(x_path, header=None).values
    Y = pandas.read_csv(y_path, header=None).values.ravel()
    return Y.tolist(), X.tolist()

# 2. normalization
def normalize(X):
    X = np.array(X, dtype=float)
    mn = X.min(axis=0)
    mx = X.max(axis=0)
    X = (X - mn) / (mx - mn + 1e-8)
    return X.tolist(), mn, mx

def normalize_test(X, mn, mx):
    X = np.array(X, dtype=float)
    X = (X - mn) / (mx - mn + 1e-8)
    return X.tolist()
```

2. **SVM algorithms:** All SVM models are implemented using libsvm library.

- a. **Linear Kernel** The Linear kernel computes similarity using the dot product. This creates a linear decision boundary between classes.

$$K(x_i, x_j) = x_i^T x_j$$

- b. **The Polynomial kernel** introduces nonlinear decision boundaries using below formula.

$$K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$$

- c. **Radial Basis Function (RBF) Kernel:** The RBF kernel measures similarity based on Euclidean distance:

$$K(x_i, x_j) = e^{-\gamma \|x_i - x_j\|^2}$$

Smaller values of γ create smoother decision boundaries, while larger values create more complex boundaries.

```
print("\nLinear SVM")
model_linear = svm_train(Y_train, X_train, '-t 0 -c 1 -q')
_, acc_linear, _ = svm_predict(Y_test, X_test, model_linear)

print("\nPolynomial SVM")
model_poly = svm_train(Y_train, X_train, '-t 1 -c 1 -d 3 -g 1 -r 1 -q')
_, acc_polynomial, _ = svm_predict(Y_test, X_test, model_poly)

print("\nRBF SVM model")
model_rbf = svm_train(Y_train, X_train, '-t 2 -c 1 -g 0.001 -q')
_, acc_rbf, _ = svm_predict(Y_test, X_test, model_rbf)
#Compariosn of above 3
print("Accuracy comparison of Linear, Polynomial and RBF models")
print(f'Linear SVM : {acc_linear[0]}\nPolynomial SVM: {acc_polynomial[0]}\nRadial basis SVM: {acc_rbf[0]}\n')
```

II. Part 2 : C-SVC using multiple C and γ values:

A function `grid_search()` was implemented to find the best combination of C and γ for all 3 Linear, RBF and Polynomial SVM models.

```

# 4. grid search for various values of C and gamma
def grid_search(X, Y, kernel_type):
    C_list = [0.1, 1, 10, 100]
    gamma_list = [1e-3, 1e-4, 1e-5]
    best_acc = 0
    best_params = None
    folds = kfold_split(X, Y, k=3)
    for C in C_list:
        # Linear
        if kernel_type == 0:
            accs = []
            for tr_idx, val_idx in folds:
                X_tr = [X[i] for i in tr_idx]
                Y_tr = [Y[i] for i in tr_idx]
                X_val = [X[i] for i in val_idx]
                Y_val = [Y[i] for i in val_idx]
                model = svm_train(Y_tr, X_tr, f'-t 0 -c {C} -q')
                _, acc, _ = svm_predict(Y_val, X_val, model, options='-q')
                accs.append(acc[0])
            mean_acc = np.mean(accs)
            print(f"Linear -> C={C}, acc={mean_acc}")
            if mean_acc > best_acc:
                best_acc = mean_acc
                best_params = C
        # polynomial/RBF
        else:
            for g in gamma_list:
                accs = []
                for tr_idx, val_idx in folds:

```

The values of $C[0.1, 1, 10, 100]$ and $\gamma[1e-3, 1e-4, 1e-5]$ were used to find the best performing accuracy.

A function `kfold_split()` of data was used. Eg: $k=3$ where 2 folds of data was used to train and 1 fold data was used to test the model. This process was repeated 3 times so that each fold became the validation set once.

```

def kfold_split(X, Y, k=3):
    X = np.array(X)
    Y = np.array(Y)
    index = np.arange(len(X))
    np.random.shuffle(index)
    fold_size = len(X) // k
    folds = []
    for i in range(k):
        if i == k - 1:
            val_index = index[i*fold_size:]
        else:
            val_index = index[i*fold_size:(i+1)*fold_size]
        train_index = np.setdiff1d(index, val_index)
        folds.append((train_index, val_index))

    return folds

```

Grid Search Process: The main function first calls the `grid_search()` method, which trains the model for every combination of CCC and γ gamma. For each combination:

1. The model is trained using training folds.
2. Validation accuracy is computed

3. Average cross-validation accuracy is calculated
4. The best-performing hyperparameters are selected: Once the best-performing values of C and γ are found, the model is retrained using the full training dataset and evaluated using the testing dataset. The final accuracies were stored in a `results[]`.

```
# PART 2: grid search for (C,gamma)
results=[]
print("\n--- TUNING LINEAR SVM ---")
best_C_linear, _ = grid_search(X_train, Y_train, 0)
model_linear = svm_train(Y_train,X_train,f'-t 0 -c {best_C_linear} -q')
_, acc_linear, _ = svm_predict(Y_test,X_test,model_linear)
results.append(["Linear", acc_linear[0],best_C_linear])

print("\n--- TUNING POLY SVM ---")
(best_C_poly, best_g_poly), _ = grid_search(X_train,Y_train,1)
model_poly = svm_train(Y_train,X_train,f'-t 1 -c {best_C_poly} -g {best_g_poly} -d 3 -r 1 -q')
_, acc_poly, _ = svm_predict(Y_test,X_test,model_poly)
results.append(["Polynomial", acc_poly[0],best_C_poly,best_g_poly])

print("\n--- TUNING RBF SVM ---")
(best_C_rbf, best_g_rbf), _ = grid_search(X_train,Y_train,2)
model_rbf = svm_train(Y_train,X_train,f'-t 2 -c {best_C_rbf} -g {best_g_rbf} -q')
_, acc_rbf, _ = svm_predict(Y_test,X_test,model_rbf)
results.append(["RBF", acc_rbf[0],best_C_rbf,best_g_rbf])
```

III. Part 3: Custom Kernel (Linear +RBF):

A custom kernel was implemented by combining the Linear and RBF kernels.

The combined kernel formula is:

$$K_{combined}(x_i, x_j) = \alpha K_{linear}(x_i, x_j) + (1 - \alpha) K_{RBF}(x_i, x_j)$$

```
def combined_kernel(x1, x2, alpha=0.5, gamma=0.001):
    return alpha * linear_kernel(x1, x2) + (1 - alpha) * rbf_kernel(x1, x2, gamma)
```

The kernel matrix was precomputed before training because LIBSVM requires custom kernels in precomputed format.

IV. Experimental Settings and Result:

A. Part 1:

Kernel Type	Hyperparameters	Accuracy
Linear SVM	C=1	95.16%
Polynomial SVM	C=1, $\gamma=1$	96.56%
Radial Basis SVM	C=1, $\gamma = 0.001$	95.08%

B. Part 2:

Kernel Type	Best Hyperparameters	Cross Validation Accuracy	Best Accuracy
Linear SVM	$C=0.1$	95.84%	96.79%
Polynomial SVM	$C=10, \gamma=0.001$	96.56%	96.87%
Radial Basis SVM	$C=10, \gamma = 0.001$	96.08%	97.14%

C. Custom Linear + RBF:

Kernel Type	Hyperparameters	Accuracy
Linear +RBF	$C=1, \alpha= 0.5, \gamma = 0.001$	95.16%

V. Conclusion and Discussion:

1. In this implementation, Support Vector Machine classifiers with Linear, Polynomial, RBF and custom kernels were applied to handwritten digit classification on the MNIST dataset.
2. The results show that nonlinear kernels perform significantly better than the Linear kernel because handwritten digit data is not linearly separable. Among the default models, the Polynomial kernel achieved the highest accuracy. After hyperparameter tuning using Grid Search and 3-Fold Cross Validation, the RBF kernel achieved the best overall testing accuracy of 97.14%.
3. The experiment also demonstrated the importance of hyperparameter tuning, as selecting optimal values of C and γ significantly improved model performance. The custom Linear + RBF kernel was successfully implemented using LIBSVM's precomputed kernel functionality, although further tuning of α and γ could potentially improve its performance.
4. Overall, we demonstrate that kernel selection and hyperparameter optimization are critical factors in improving SVM classification performance on high-dimensional image datasets such as MNIST.